CIRCL
Computer Incident
Response Center
Luxembourg

Scaling the Integration

MISP Integration Workshop

Why query-per-event doesn't scale

Tune the trigger

Cache MISP responses

Pull model with a CDB list

Insulate the MISP instance

TLP:CLEAR

The starting point

query-per-event integration.

- Every matching alert is handed to `custom-misp.py`.
- The script makes a **synchronous MISP REST call** per IOC.
- Fine for a few lab agents — it does **not** scale.

> Lookup volume tracks **alert** volume, not the number of real matches.

```
<ossec_config>
    <integration>
    <name>custom-misp.py</name>
    <group>syscheck</group>
    <hook_url>https://YOUR_MISP_IP:PORT/</hook_url>
    <api_key>YOUR_API_KEY</api_key>
    <alert_format>json</alert_format>
    </integration>
</ossec_config>
```

Why it breaks down

A single MISP instance on the hot path of every event.

- Every FIM and Suricata record becomes one+ MISP queries — mostly **no match**.
- A few hundred agents can produce **thousands of lookups/second**.
- Under load: rising API latency, a backed-up `wazuh-integratord` queue, dropped alerts, and MISP going unresponsive for analysts.

> Fixes go cheapest → most robust; in production you combine several.

Reduce the lookups

The cheapest win is not making the query at all.

- **Tighten the trigger** — narrow the `<group>` filter; only high-value alerts.
- **Reduce noise at the source** — scope FIM directories, tune Suricata logging.
- **Prefer specific IOC types** — hashes/domains over raw IPs; drop what you don't act on.

```
<syscheck>
  <!-- report added files -->
  <alert_new_files>yes</alert_new_files>
  <!-- scope dirs + restrict to executables -->
  <directories check_all="yes" realtime="yes"
    restrict="\.exe$|\.dll$|\.sh$|\.bin$">/usr/local/bin
  </directories>
</syscheck>
```

> Lowers load, but keeps the per-event, single-instance dependency.

Cache MISP responses

The same IOCs recur constantly across agents.

- Add a local cache (Redis or on-disk KV) in front of the MISP call.
- **Cache negative results too** — they dominate — with a short TTL.
- Or put a **caching reverse proxy** in front of `/attributes/restSearch`.

> A flood of lookups for one benign value hits MISP once, not thousands of times.

Switch to a pull model (recommended)

Export IOCs from MISP; match locally on the manager with a CDB list.

- Removes MISP from the hot path — matching is an **in-memory lookup**.
- **Zero API calls per event**, no matter how many agents you run.
- Reuse the PyMISP export (`get_iocs_csv.py`) from cron every 15–30 min.

> Trade-off: detections are only as fresh as your last sync.

Insulate the MISP instance

Treat MISP as a shared production service.

- Give the integration a **dedicated API user** — rate-limit, audit, revoke.
- Serve API traffic via a **caching proxy**; use a **read-only replica** for lookups.
- Publish IOCs as a **feed/export** that consumers pull, not live API hits.

> Keep the primary instance responsive for analysts on the web UI.

Questions?

TLP:CLEAR